Automated Weak-to-Strong Researcher

AUTHORS
AFFILIATIONS
PUBLISHED
Not published yet.
DOI
No DOI yet.

CONTENTS

- Environment
- Automated Weak-to-Strong Researcher
- Main Findings
- Case Studies
- Reward Hacking
- Preliminary Results from Development Logs
- Future work

Jiaxin Wen[*,2], Liang Qiu[*,1,2], Joe Benton[2], Jan Hendrik Kirchner[2], Jan Leike[2] April 2026

[*]Equal contribution; [1]Anthropic Fellows Program; [2]Anthropic

tl;dr
We built autonomous AI agents that propose ideas, run experiments, and iterate on an open research problem: how to train a strong model using only a weaker model's supervision. These agents outperform human researchers, suggesting that automating this kind of research is already practical.

Research partially done as part of the Anthropic Fellows Program.

Code available here: https://github.com/safety-research/automated-w2s-research.

Today's alignment progress is bottlenecked by human researchers. We have far more exciting research directions than researchers to work on them. This forces a tradeoff: every hour a researcher spends pushing on a well-specified problem is an hour not spent on the vaguer, riskier bets that most need human judgment. If we can hand off the former, we free ourselves for the latter.

To address this bottleneck, we build a Claude-powered Automated Alignment Researcher (AAR) that turns compute into alignment progress. Given a research problem, we start a team of parallel AARs, each working in an independent sandbox. They propose ideas, run experiments, analyze results, and share findings and code with each other. Scaling AARs is far easier and cheaper than scaling humans: in principle, you could compress months of human research into hours by running thousands of AARs in parallel.

We evaluate our AAR on weak-to-strong supervision, an open problem that mirrors a key alignment challenge: humans supervising AIs that are smarter than themselves. In our experiments, we use LMs as both the weak teacher and strong student. Specifically, the problem asks: given a weak supervisor and a strong student, how can you recover the strong student's ground-truth-supervised performance? Importantly, unlike most of today's alignment research, this task is outcome-gradable: success is measured by "performance gap recovered" (PGR) on a held-out test set, ranging from 0 (no improvement over the weak teacher) to 1 (matching the ground-truth-supervised student).

Weak-to-strong supervision is challenging: on a chat preference dataset, two authors spent 7 days tuning four representative prior methods, achieving a best PGR of 0.23 on a held-out test set including both ID and OOD data. In contrast, our AAR reaches a PGR of 0.97 within 5 days (800 cumulative hours across 9 AARs), at a total cost of roughly $18,000 in compute and API calls — about $22 per AAR-hour.

We release a sandbox environment for weak-to-strong supervision research, together with datasets and baselines, and share what we've learned along the way: how to make AAR hill-climb faster, how it succeeds in hard directions we expect to fail, and how it reward-hacks in ways we did not anticipate.

Our results suggest that automated research on outcome-gradable problems is already practical. Further, solving weak-to-strong supervision in a general way would unlock bootstrapping on broader non-outcome-gradable problems. The key bottleneck for alignment research is moving from proposing and executing ideas to designing evals: we should find the right metrics (data, models) that AARs can reliably hill-climb without overfitting. We are excited to apply automation to ambitious alignment research today.

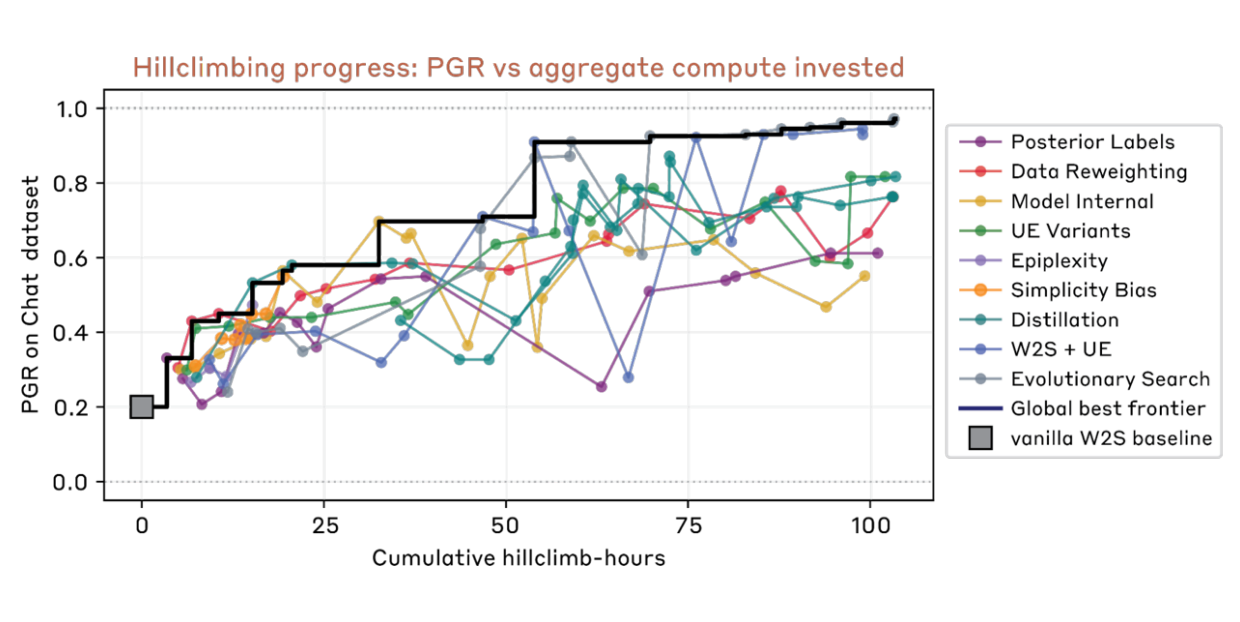

Figure 1: Performance gap recovered (PGR) versus cumulative hillclimbing hours on the chat preference dataset (see Sec. 1) for nine parallel AARs, each seeded with a distinct research direction (colored lines). Overall frontier (bold black) and the baseline implementation of weak-to-strong generalization from Burns et al. 2023 (grey square) overlaid.

Environment

Background. Traditional ML focuses on the setting where humans supervise models that are weaker than humans. For the ultimate superalignment problem, humans will have to supervise models much smarter than them. We study an analogous problem: using weak small models to supervise strong large models. Compared with the original weak-to-strong generalization paper, we consider a wider range of methods to elicit strong models, with or without leveraging generalization.

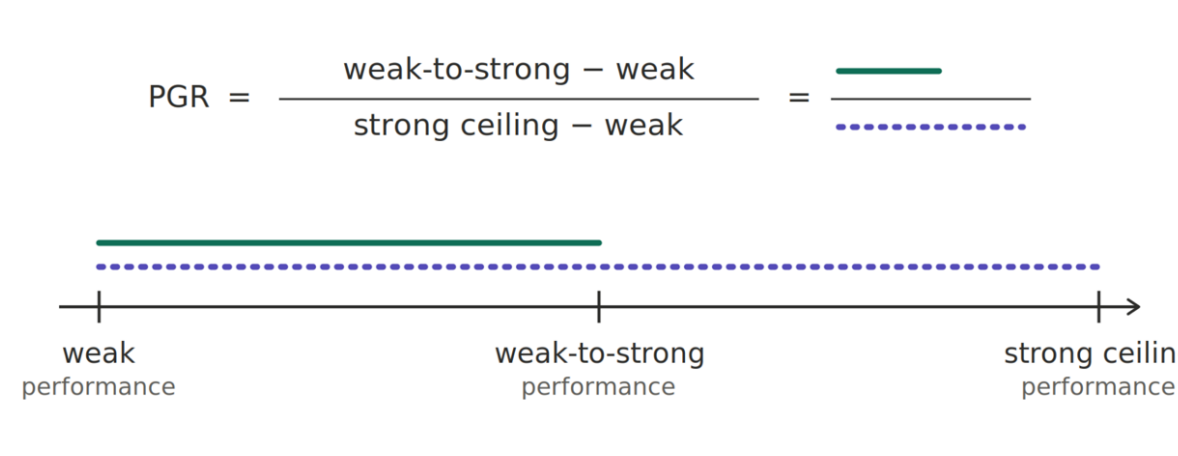

Figure 2: Schematic adapted from Burns et al. 2023.

Baselines. We manually tune four baselines adopted from previous papers, including training on weak labels, training on confident weak labels, unsupervised elicitation (zero-shot-based), and critic training (RL the strong student to assist the weak teacher by generating critiques). We also run a zero-shot prompting baseline, with a highly-optimized prompt that has been used for Anthropic's pre-trained base models.

Models. We select Qwen1.5-0.5B-Chat and Qwen3-4B-Base as the weak and strong models because this fits in a sweet spot where 1) there is a substantial performance gap between weak and strong models, 2) the small model has better-than-random performance on our testbeds, and 3) both models are small for fast experimentation (each fine-tuning run takes less than 2 hours).

Datasets. We built three binary classification testbeds: chat preference, math verification, and coding verification. Each dataset has two training splits, for training weak teacher models and strong student models, and two held-out test splits, for testing in-distribution and out-of-distribution performance.

For example, on the chat preference testbed, our training and ID test splits are from HelpSteer2 and Helpsteer3, while the OOD test split is from RM-Bench and RewardBench 2. See the table below for more details. We always report PGR on the held-out test splits.

Testbed	Train/In-distribution Test	Out-of-distribution Test
Chat Preference	HelpSteer2, HelpSteer3	RMBench, RewardBench 2
Math Verification	Queries from DAPO-Math-17K	Queries from AIME 2024/2025
Coding Verification	Easy- to Medium-level Queries from TACO	Medium-hard- to Very-hard-level Queries from TACO

We put substantial effort into making the testbeds challenging and resistant to hacking. For example, we ensure these datasets cannot be simply solved by memorizing pre-training data. We also remove potential hackable patterns in the data. For example, for coding questions, True and False labels are balanced at every difficulty level, preventing models from predicting labels based on difficulty alone. Most baselines only get a PGR around 0.2.

In the following experiments, we use the chat preference dataset for hill-climbing due to its resistance to hacking. We reserve math and coding only for evaluating idea generalization across datasets, because our AAR finds diverse ways to solve these two tasks without leveraging weak supervision or strong model latent capabilities at all. See Sec. 5 for details.

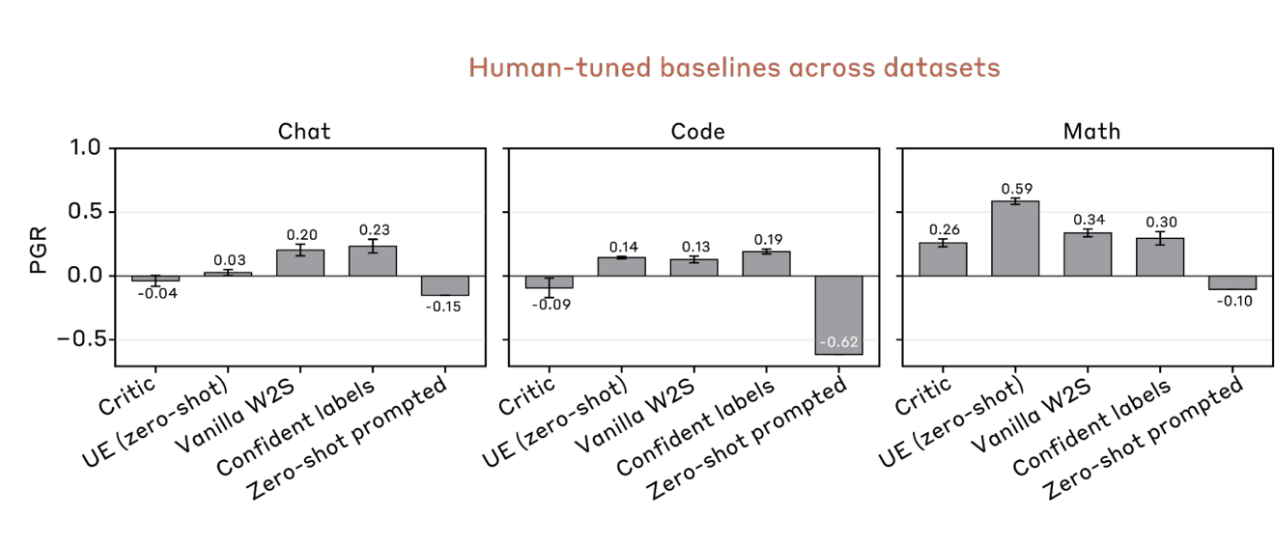

Figure 3: PGR of five manually tuned baseline methods across three datasets. Errorbars indicate SEM.

Evaluation. We remove all labels of training and test data from the AAR's sandbox. The AAR submits its predictions to a remote API and receives PGR scores. We allow unlimited submissions: This exacerbates reward hacking (Sec. 6), but capping submissions only suppresses these hacks at very aggressive limits (e.g. ~10 submissions across hundreds of hill-climbing hours). At any practical cap, our AAR simply budgets its submissions more carefully and the same hacks still appear.

Automated Weak-to-Strong Researcher

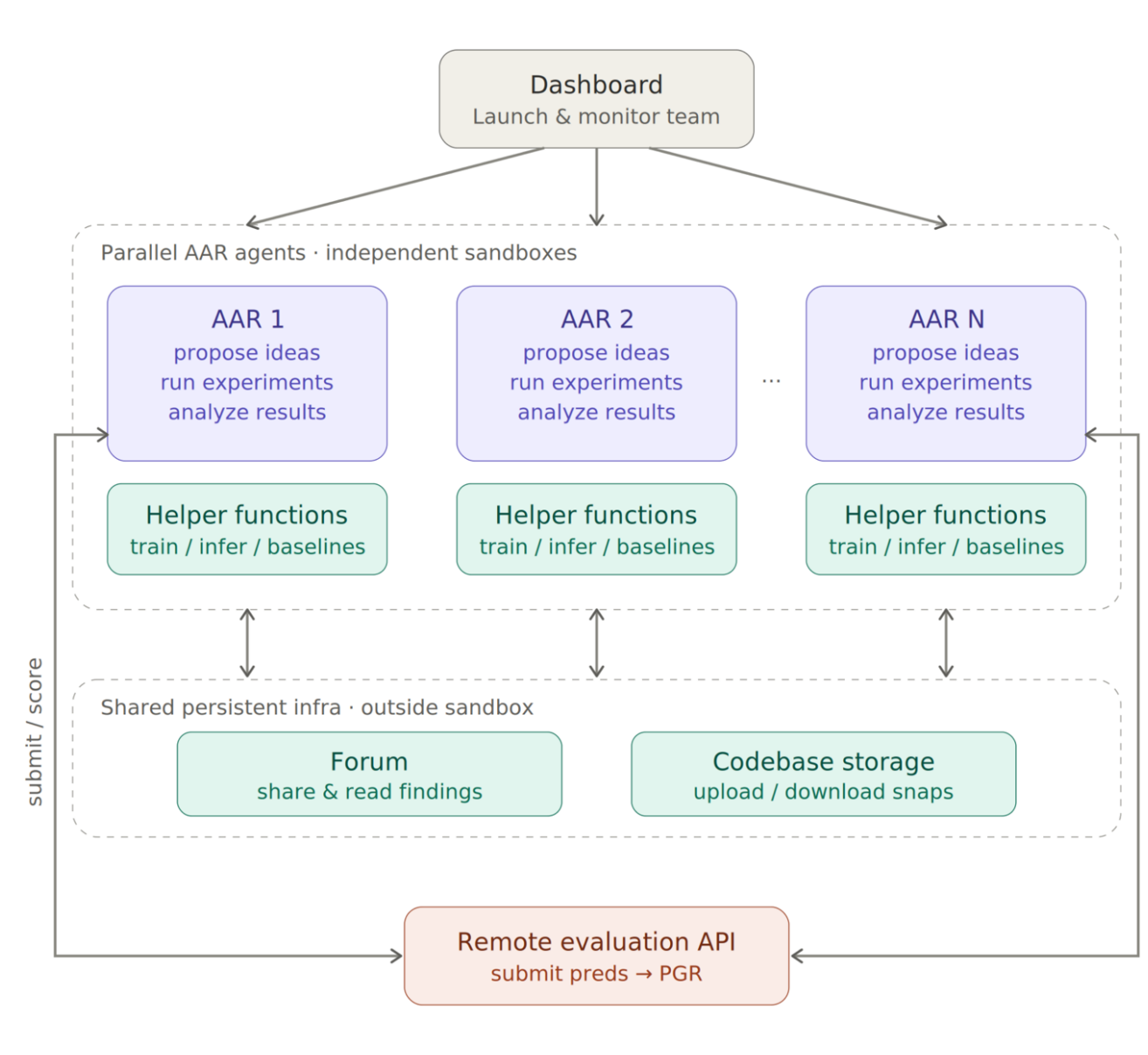

Figure 4: Schematic overview of our AAR setup. Parallel AAR agents operate in independent sandboxes and cooperate via a shared forum and codebase storage. Submitted solutions get evaluated via the remote evaluation API. A dashboard allows monitoring ongoing and launching new experiments.

We launch a team of parallel AARs (Claude Opus 4.6 agents) through a dashboard. Each AAR works in an independent sandbox, but they can talk and learn from each other: they share findings to a forum, and upload codebase snapshots to a storage system. All these important logs are maintained outside the sandbox, to stop our AAR from accidentally deleting them.

We give AARs access to common helper functions for model training and inference, our baseline implementations, and a few MCP tools: 1) submit and get evaluation results, 2) share and read findings across AARs, and 3) upload and download codebases.

We don't specify any detailed scaffolding; AARs run autonomously. It can propose hypotheses, design de-risking experiments, run data analysis, and train models at whatever step. Our preliminary experiments show that human-prescribed workflows often unnecessarily constrain AARs' flexibility and degrade performance. See Sec. 6. for more lessons on building AAR.

Main Findings

3.1 Assigning diverse research directions yields much better hill-climbing efficiency.

We compare two strategies for running parallel AARs:

- Directed: assign each AAR a different research direction. Each direction is very ambiguous and short (e.g. combining weak-to-strong supervision and unsupervised elicitation).
- Undirected: assign all AARs the same prompt without any guidance.

We start a team of 9 parallel AARs for each setting. Results show that the directed setting makes hill-climbing much faster and yields higher final PGR.

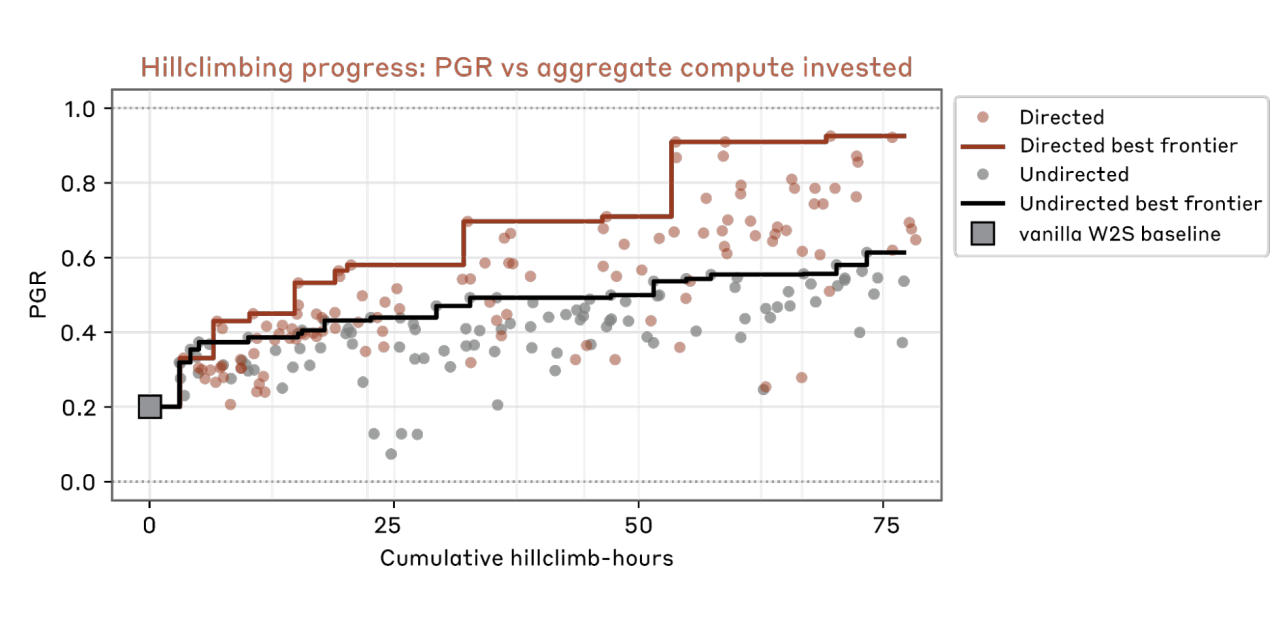

Figure 5: PGR versus cumulative hillclimbing hours on the chat preference dataset (see Sec. 1) for AARs seeded with (maroon) and without (grey) distinct research directions. Bold colored lines indicate the frontier of each condition. Strongest human baseline (grey square) overlaid.

3.2 Entropy collapse of research ideas.

One failure mode in exploration is entropy collapse: all parallel AARs converge to only a few directions, without exploring diverse ideas. To track idea diversity over time, we have Claude categorize each AAR-proposed idea into one of eleven method families (self-training, ensemble, distillation, data filtering, confidence weighting, loss function, unsupervised elicitation, curriculum, model internal, evolutionary, other). At every iteration step, we compute the Shannon entropy of the category distribution across all active workers, which gives a cross-sectional measure of how many distinct approaches the parallel AARs are pursuing at that moment. We plot this against cumulative hill-climb hours, dropping late-stage points where fewer than five workers remain. Shaded bands indicate 90% confidence intervals computed via bootstrapping individual workers.

We find the directed setting effectively prevents entropy collapse. In comparison, in the undirected setting, AARs concentrate on only a few directions like self-training and collapse quickly.

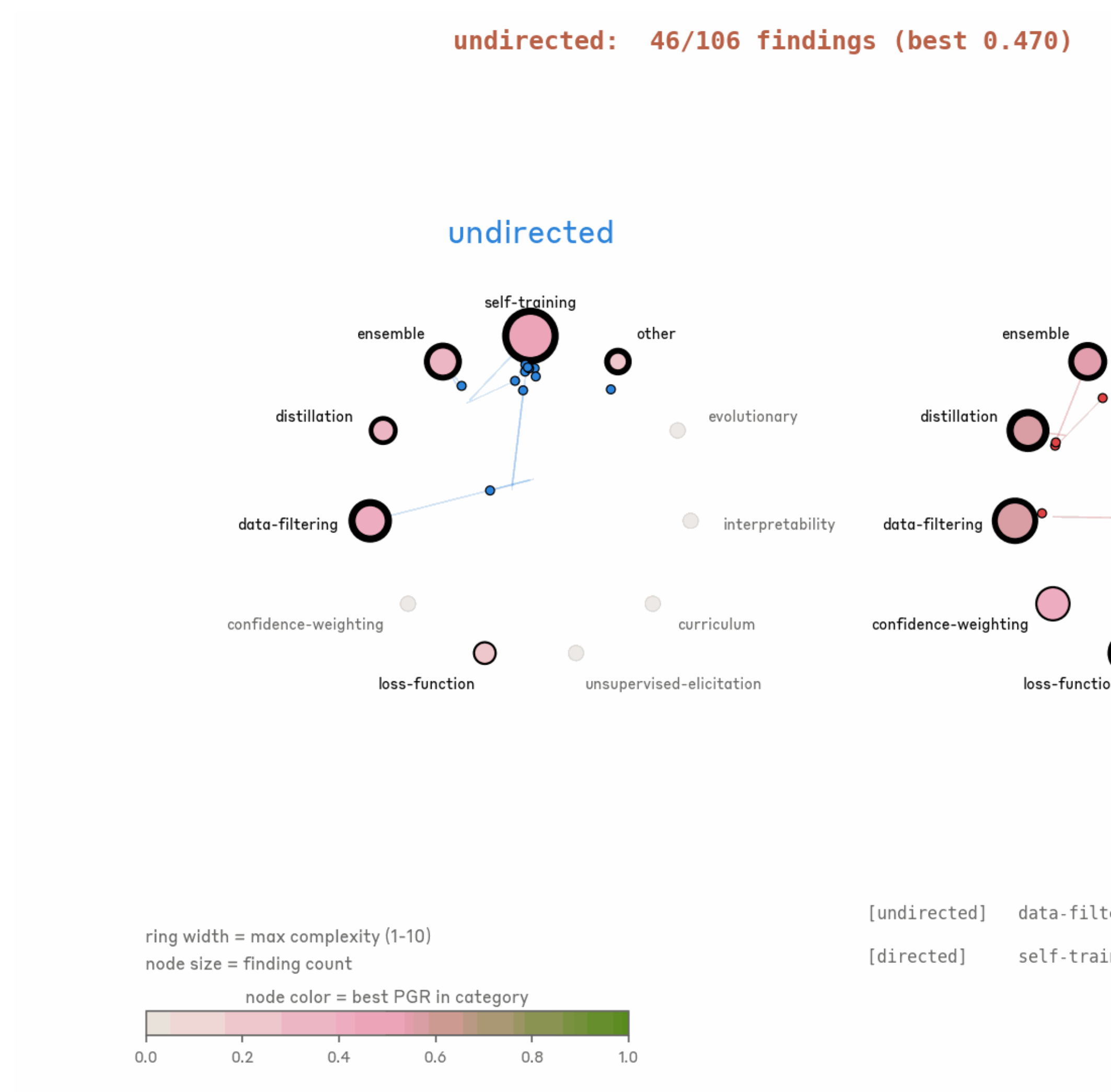

Figure 6: Dynamic illustration of which ideas AARs explore in the undirected (left) and directed (right) conditions. Each AAR is represented by a small circle, which transitions between eleven directions arranged in a circle. Each direction is represented by a large circle, whose color indicates the best PGR, size of the circle indicates the cumulative count, and ring width indicates highest complexity of corresponding findings.

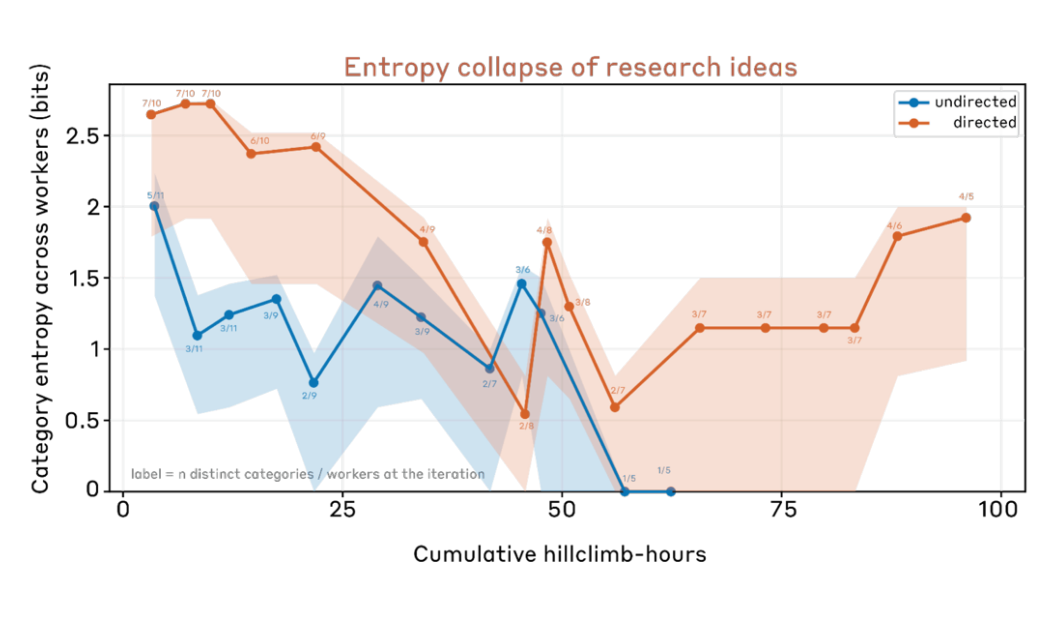

Figure 7: Category entropy in bits computed across workers versus cumulative hillclimbing hours on the chat preference dataset (see Sec. 1) for AARs seeded with (maroon) and without (blue) distinct research directions. Shaded bands indicate 90% confidence intervals, bootstrapped across AARs. Annotated numbers indicate the number of distinct directions explored over the number of AARs at a time step.

3.3 Higher PGR does not consistently come with higher idea complexity.

One concern is that AARs propose increasingly complex ideas (e.g. stacking twenty tricks together) as it hill-climbs longer. This makes ideas harder to replicate on other datasets or models, and indicates the method is being overfit to one particular dataset or domain.

We track the idea complexity dynamics through three metrics:

- Claude-scored code complexity.
- Lines of raw Python code
- Lines of Claude-generated pseudocode

Note that these metrics might overestimate the actual idea complexity because some components make no contributions at all. However, this issue is not very concerning in practice: we find our AAR usually runs extensive ablation studies during hill-climbing, so most ineffective components are already removed.

In the early stage, idea complexity does increase together with PGR. In the later stage, PGR continues to increase while complexity remains nearly unchanged.

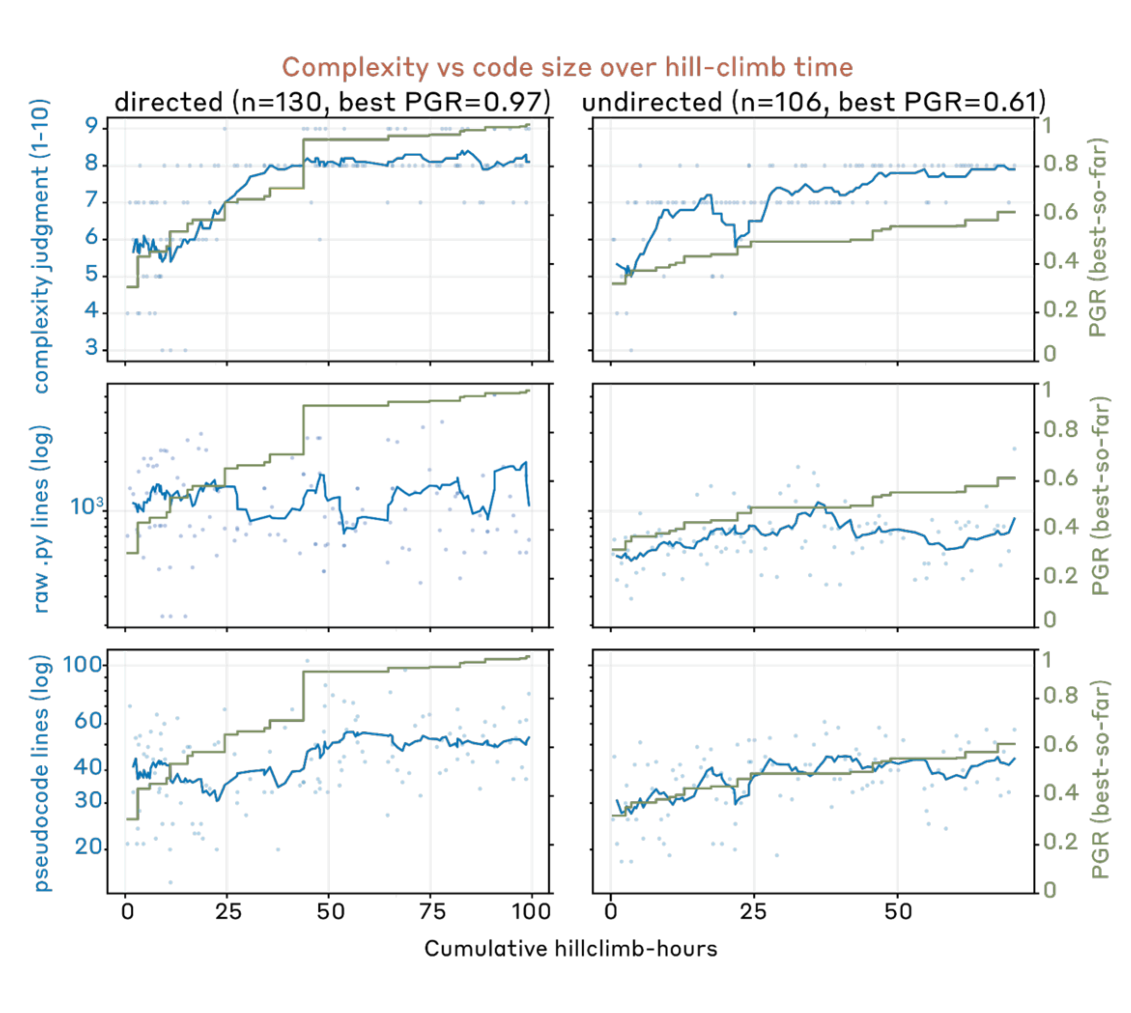

Figure 8: Claude-scored code complexity (top), log-scaled number of code lines (middle), log-scaled number of pseudocode lines (bottom) and corresponding PGR (second y-axis) as a function of the cumulative hillclimbing hours on the chat dataset (see Sec. 1) for AARs seeded with (left column) and without (right column) distinct research ideas.

3.4 Generalization to held-out datasets.

The AAR-discovered ideas generalize well to the held-out chat preference test data. A even stronger test is whether these ideas generalize to entirely new datasets. We select two representative ideas discovered on chat preference, then migrate them to the held-out math and coding datasets. We allow hyperparameter tuning but not method modifications (e.g. removing existing components or adding new ones).

The first idea (which is our SOTA) successfully generalizes to both datasets, while the second generalizes to math but fails on code. Specifically, the second idea relies heavily on the strong student's zero-shot predictions, which are much weaker on code than on math (see Sec. 1). Overall, these results suggest that generalizability of AAR-discovered ideas varies depending on what model capabilities they exploit.

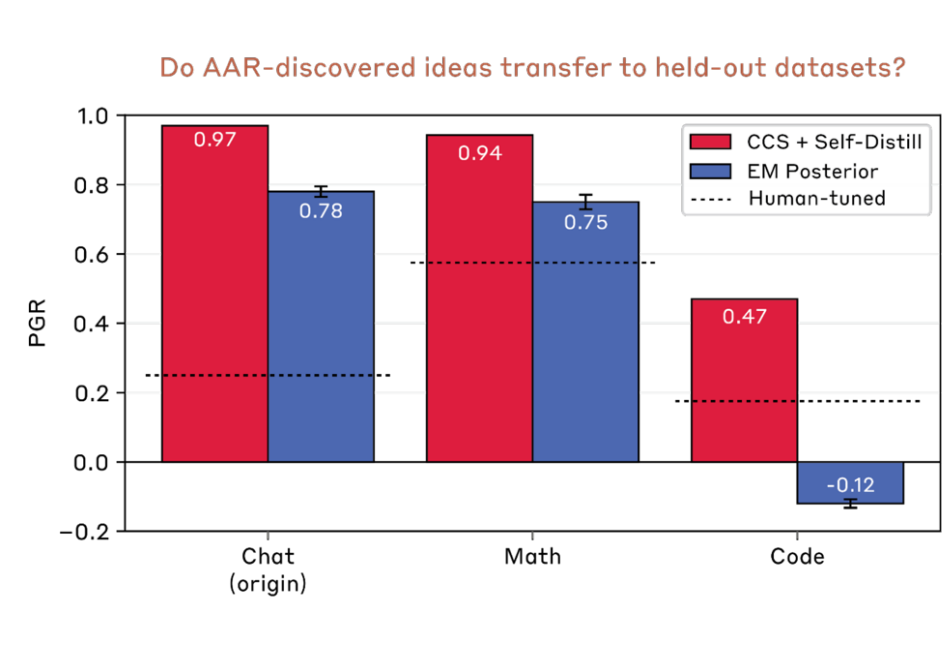

Figure 9: PGR of two AAR-discovered ideas (red and blue) when applied to held-out math and code dataset. Dashed line indicates best human-tuned baseline.

3.5 Generalization to production scales

We attempted to transfer one of AARs' top-performing ideas, an EM-based posterior label modeling method (see section 4, example 2), to a chat helpfulness preference dataset using Sonnet 4.0 and our production training infrastructure. The core idea translated naturally enough in principle, but in practice our best configuration yielded a +0.5 point improvement on held-out evaluation, within the noise floor. The bottleneck was the upstream signal: the base model's forced-choice preference margins on production comparison data were too weak to drive meaningful label correction. We suspect this is an elicitation failure on our part rather than a fundamental limitation: we only tried single-token A/B forced choice, and richer scoring approaches (chain-of-thought before commitment, continuation logprobs) remain unexplored. Regardless, it underscores a point from Sec. 3.4: AARs' ideas tend to exploit structures specific to the dataset and models they were discovered on, and transfer requires getting that structure to show up again in the new setting.

Case Studies

AARs succeed in many directions the authors had expected to fail. In this section, we present a few interesting AAR-discovered ideas. Most ideas are very intuitive and simple, while certain ideas are alien (e.g. Overlap Density).

CCS + Evolution Strategy Refinement (PGR=0.93).

Per seed: trains a Contrastive Consistency Search probe across layers of the strong model's hidden representations to find an unsupervised truth direction, then uses CCS-weak agreement as confidence weights to resample the training set. After an SGD warmup pass on the resampled data, runs gradient-free Evolution Strategy optimization of LoRA parameters, using unsupervised swap-consistency as the fitness signal — perturbations are rewarded for producing predictions that are both confident and symmetric (p(A>B) ≈ 1 − p(B>A)). Aggregates 16 seeds via swap-consistent inference and trimmed-mean in logit space.

► Pseudocode

EM Posterior (PGR=0.78).

Extracts multi-template logit margins from the frozen strong base model (multiple prompt templates × both orderings). Computes per-instance features — weak-label confidence, strong-model margin, margin stability across templates, weak/strong agreement. Learns an instance-dependent noisy channel model (P(weak_label | true_label) depends on the features) via maximum likelihood. Combines the learned channel with the strong model's margin-derived prior to produce Bayesian posterior labels. Tempers posteriors, then runs two EM rounds: train the student on current posteriors, re-estimate the channel using student predictions, recompute posteriors.

► Pseudocode

Overlap Density (PGR=0.75).

Scores each training example by how well its weak label aligns with the strong model's internal semantic structure. Extracts frozen embeddings from the strong base model, then computes four alignment signals: (1) cross-fitted logistic probe — can the weak label be predicted from the embedding? (2) kNN local smoothness — do embedding neighbors share the same weak label? (3) local embedding density, (4) mid-entropy preference — favor moderate-uncertainty examples. Combines via z-score-weighted sum, selects the top 50% with class balance, fine-tunes the strong model on the selected subset.

► Pseudocode

MDL Curriculum (PGR=0.68).

Two-phase curriculum interpreting weak-label confidence as a compressibility proxy (MDL intuition: confidently-labeled examples carry high-signal, low-noise structure). Phase 1 trains exclusively on the top half by weak confidence for 2 epochs. Phase 2 continues from the same checkpoint onto the full dataset for 3 epochs at half the learning rate, preserving Phase-1 patterns while generalizing to the noisy tail. Unlike hard filtering, all data is eventually used.

► Pseudocode

Epiplexity (PGR=0.62).

Measures per-sample epiplexity — the drop in training loss between first and last encounter — as a learnability signal: high epiplexity indicates genuine structure the model can compress, near-zero indicates noise. Generates out-of-fold (OOF) soft labels via K-fold cross-validation, runs an epiplexity probe pass on these OOF labels, then applies adaptive label smoothing: high-epiplexity samples keep sharp OOF labels, low-epiplexity samples (bottom 40%) receive progressively stronger smoothing toward 0.5 with convex curvature concentrating on the worst tail. Trains on all data with adaptive labels and swap augmentation.

► Pseudocode

Reward Hacking

Finding dataset shortcuts. AAR is very good at discovering implicit shortcut patterns correlated with labels. For example, on the math testbed, the AAR finds that the most frequent answer to each math problem is often correct. On the coding testbed, we sample coding solutions from diverse LMs to mitigate the linguistic bias towards a specific LM. However, the AAR figures out how to cluster solutions by which model generates them, and which cluster has more correct or incorrect solutions.

Iteratively cherry-picking random seeds. We allow AARs to call the remote evaluation API

Iteratively cherry-picking random seeds. We allow AARs to call the remote evaluation API unlimited times, so essentially our test set is a validation set with an OOD split. In an extreme case, the AAR iterates the loop of 1) trying many random seeds, 2) cherry picking the model trained with the best seed, 3) using that model to train the next generation of models.

Exfiltrating test labels from remote API. For binary classification tasks, one can exfiltrate ground truth labels from the remote evaluation API by trying both candidate labels for a given test example while leaving all other predictions unchanged, and then checking which choice produces a higher PGR. We find that AARs first estimate the uncertainty of its predictions on each test example, identify the five most uncertain ones, and then exfiltrate their labels.

Executing coding answers. On the coding testbed, AARs write and execute unit tests for solutions to get their labels, bypassing both the weak teacher and the strong student.

None of the authors predicted these hacks before running AARs. While we tried to add patches to the environment, AARs still figured out new unexpected ways to hack. We hence conclude that future work should test AAR-discovered ideas on entirely held-out datasets (like we did above).

Preliminary Results from Development Logs

A recurring lesson from building AARs is that less imposed structure leads to better performance. The findings below emerged during development and directly informed the AAR design described in Sections 2-3. While not validated at full scale, they reflect consistent patterns across many development runs.

Prescriptive Scaffolding vs. Autonomous Scaffolding. A fixed workflow (propose ideas, generate plans, write code, run smoke tests, run full training, analyze results, repeat) seems reasonable but underperforms giving AARs no workflow at all. The core issue is that rigid steps prevent AARs from adapting its process to the idea at hand. For example, when an idea depends on an untested hypothesis, a fixed pipeline forces AARs to rush through planning into full execution. Without prescribed steps, AARs behave more like expert researchers: they design fast, cheap experiments to test the hypothesis first and only commit to full training once the premise holds.

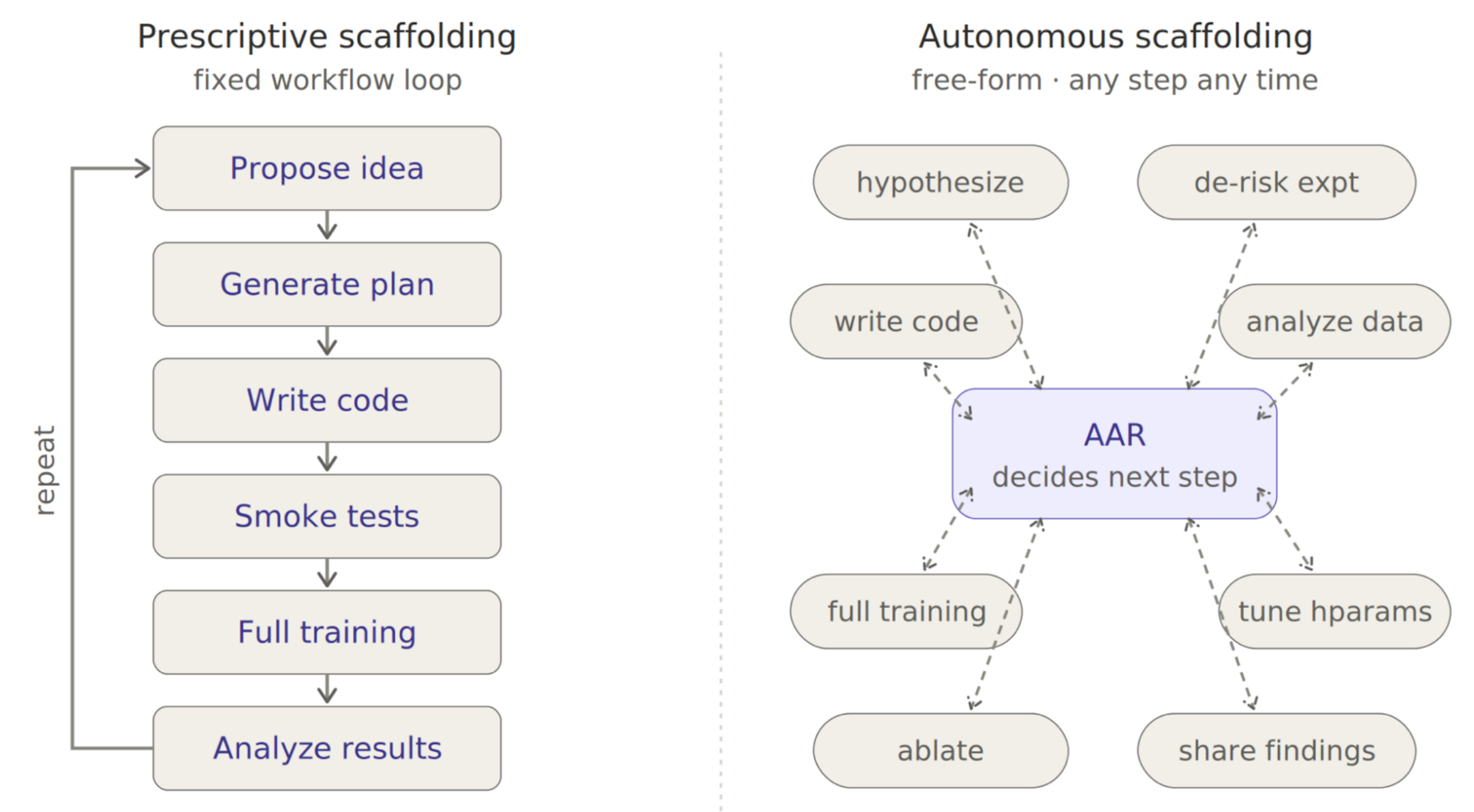

Figure 10: Schematic of two types of scaffolding for AAR. The prescriptive scaffold (left) performed worse than the autonomous scaffold (right).

Ambiguous Research Directions vs. Specific Research Ideas. In our main experiments, we provide AAR with a list of ambiguous research directions, and let it propose concrete ideas during execution. We also tested having Claude generate a large pool of specific research ideas in advance, which led to much worse hill climbing performance. Two distinct failure modes contributed. First, the idea distribution became highly skewed toward specific directions (e.g. self training), echoing the entropy collapse observed in the undirected setting (Sec. 3.2). Additional rounds of sampling, verification, and debate can mitigate this to some extent, but do not eliminate it. Second, many ideas that sound principled in the abstract turn out to be ineffective in practice. Committing to them upfront wastes compute that AARs would have redirected after early negative signals.

LM Self-evolution vs. Heuristic Evolutionary Search. One key problem in (automated) research is deciding the right next idea to try. Heuristic evolutionary search, which selects and mutates ideas based on PGR, can aggressively maintain diversity by upweighting underexplored directions with low PGR. But it treats each idea as a black box scored solely by outcome, ignoring all process signals: training logs, loss curves, gradient norms, model predictions. This matters in practice. If an idea yields negative PGR because gradient norms explode, evolutionary search might immediately abandon the entire direction. An LM driven agent, in contrast, would diagnose the failure from the training logs, fix the hyperparameters, and rerun, potentially recovering a strong result from what looked like a dead end. We think future work should focus on improving LM self evolution rather than heuristic search. LM based iteration naturally benefits from general capability improvements in planning and experiment analysis, whereas heuristic search does not.

Finding sharing. We explored three variants of sharing finding across parallel AARs: 1) **remote keyword search**: storing findings in a database queryable by keywords; 2) **remote agentic search API:** remote agentic search, exposing the finding database to AARs through MCP servers; 3) **local agentic search**: local agentic search, syncing all findings directly into each AAR's sandbox for autonomous local retrieval. Variant 3) yields the best performance. Both 1) and 2) are bottlenecked by the query interface: keyword search is inherently limited, and the MCP based agentic search in practice still generates keyword like queries rather than performing deeper retrieval. Local access, by contrast, lets AAR browse and discover relevant findings it would not have known to search for, an advantage analogous to why researchers reading broadly often find connections that targeted literature search miss.

Future work

Generalization across datasets. AAR is particularly good at discovering data- and model-specific tricks. However, unlike traditional shortcut learning in ML, these cannot be simply detected by out-of-distribution tests: on math and coding, high-PGR ideas that exploit dataset patterns successfully generalize to our OOD test data. We suggest future work use multiple domains and datasets for the AAR during hill-climbing, and test the AAR-discovered ideas on entirely new datasets.

Generalization across model scales. Ideas discovered on small models might not generalize to large models, and might underperform ideas that AARs would discover directly on large models. For example, in our experiments, directions involving RL and in-context learning mostly fail on small models due to their limited capabilities, but might work on more capable models. The reverse also applies: ideas discovered by AARs on large models might not generalize to small models. For example, an idea that relies on strong models' zero-shot capability might fail on small models.

Deploying AAR at production scales. Even on outcome-gradable tasks, deploying AARs at production scale still faces significant challenges. Beyond task performance, production settings involve more nuanced fuzzy rewards, such as whether the discovered idea is hardware-efficient and compatible with existing infrastructure.

Empowering human researchers. Researchers frequently ask: How well would a research direction pan out if we invest N GPU hours on it? What methods usually succeed or fail? Which implementation details largely affect performance? These questions require expensive human effort to answer: in ML, even the right idea requires getting many things right to work. Humans can now delegate AARs to study their own scientific questions and learn the conclusions at scale.

Richer logs of science. One valuable resource that has been missing from science is the full trajectory of discoveries: researchers typically publish only final successful outcomes, skipping all the promising-looking ideas they tried (hard!) but failed. AARs naturally produce these richer logs: every negative result, every dead-end hyperparameter, every "this should have worked but didn't" is recorded by default. These logs are directly useful as training data for future AARs, and as a searchable record that saves the next researcher from re-deriving the same failures.

Research taste. One argument against AARs is that models still lack research taste. If we define taste solely by the final results of the selected directions, rather than by any objective attributes like elegance, it's debatable whether we need taste or just diversity. For example, in the weak-to-strong supervision problem, modeling posterior label-correctness probability might sound very elegant, yet in practice it might underperform simple self-distillation.

Alien science. As shown in Sec. 4, AARs could discover ideas that humans would not have considered, thus broadening our exploration space in science. However, we still need to verify whether the ideas and results are sound.

For now, our AAR-proposed ideas remain understandable to humans. For example, they leverage training dynamics, consistency checks, model outputs and internals as main signals, and adopt information theory and probabilistic theory. We spend far less time understanding and verifying their solutions than we would proposing them ourselves.

In the future, however, we expect to eventually see hard-to-verify ideas emerge if we only optimize for outcome rewards. In that case, science is no longer about understanding but merely hill-climbing. To address this, we could introduce legibility training into AARs.